



Міністерство освіти і науки України
Національний технічний університет України “Київський політехнічний
інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №2
З дисципліни «Технології розроблення програмного забезпечення»
Тема: **«ДІАГРАМА ВАРІАНТІВ ВИКОРИСТАННЯ. СЦЕНАРІЙ
ВАРІАНТІВ ВИКОРИСТАННЯ. ДІАГРАМИ UML. ДІАГРАМИ КЛАСІВ.
КОНЦЕПТУАЛЬНА МОДЕЛЬ СИСТЕМИ»**
Система ведення нотаток

Виконав:
Студент групи ІА-24
Шкарніков А. С.

Перевірив:
Мягкий М. Ю.

Київ-2024

Тема:.....	3
Мета:	3
Завдання:.....	3
Хід роботи	4
1. Схеми прецедентів.....	4
2. Оберемо 3 прецеденти і напишемо для них сценарії використання.....	5
3. Схеми класів.....	7
4. Структура бази даних	8
Висновки:	8

Тема:

Діаграма варіантів використання. Сценарії варіантів використання. Діаграми uml. Діаграми класів. Концептуальна модель системи

Мета:

Метою даної лабораторної роботи є дослідження та практичне застосування UML-діаграм для моделювання варіантів використання системи та її концептуальної структури. Зокрема, робота зосереджується на побудові діаграм прецедентів, діаграм класів, а також створенні сценаріїв варіантів використання для системи. Вивчення цих моделей сприяє глибшому розумінню об'єктно-орієнтованого проектування та взаємодії між компонентами програмної системи.

Завдання:

В якості нотаток має бути, як мінімум, текст або зображення, додатково можна додати підтримку файлів розміром до 2Мб, збереження посилань на веб-сторінки, збереження html-тексту з коректним його відображенням. Система повинна дозволяти вести нотатки онлайн, заводити блокноти, прикріплювати до нотаток мітки, давати можливість працювати в системі одночасно багатьом користувачам і одному користувачу працювати з різних комп'ютерів. Користувач повинен мати можливість дати доступ до свого блокноту з замітками іншому користувачу з різними правами (тільки перегляд; перегляд та редагування; перегляд, редагування, додавання та видалення нотаток)

Хід роботи

1. Схема прецедентів

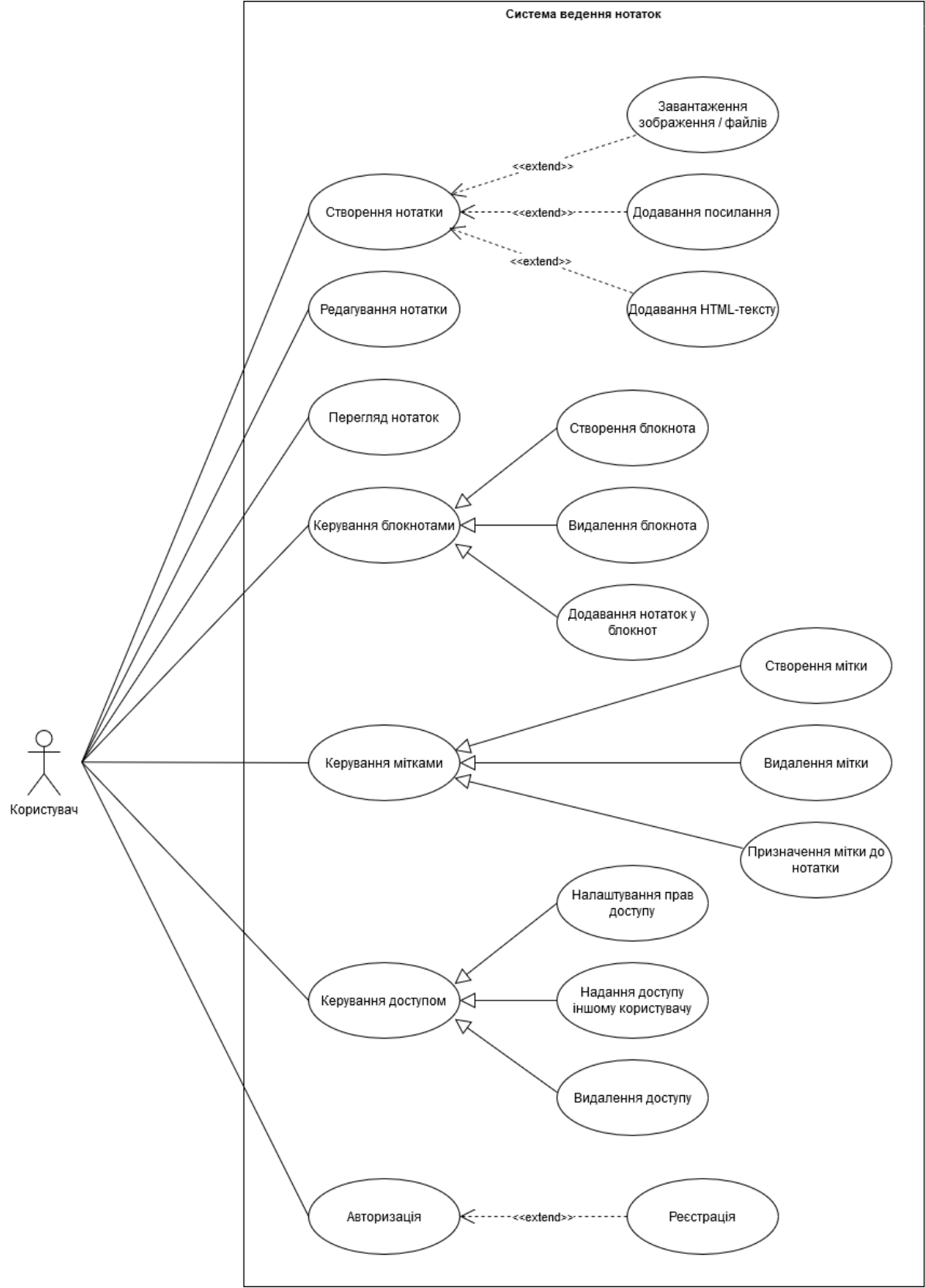


Рисунок 1-1 Діаграма прецедентів

2. Оберемо 3 прецеденти і напишемо для них сценарії використання

Сценарій використання 1: Створення нотатки

Передумови: Користувач увійшов у систему.

Післяумови: Нотатку створено та збережено в системі.

Актори: Користувач.

Опис: Цей сценарій описує процес створення нової нотатки в системі.

Основний хід подій:

1. Користувач натискає кнопку «Створити нотатку».
2. Користувач вводить назву нотатки.
3. Користувач вводить текст нотатки.
4. (Необов'язково) Користувач завантажує зображення, файл, додає посилання або вставляє HTML-текст.
5. Користувач натискає кнопку "Зберегти".
6. Система зберігає нотатку та підтверджує успішне збереження.

Винятки: Якщо зображення, файл або HTML-текст перевищує обмеження у 2Мб, система видає повідомлення про помилку.

Примітки: Нотатку можна створити, ввівши лише текст.

Сценарій використання 2: Створення блокнота

Передумови: Користувач увійшов у систему.

Післяумови: Блокнот створено та додано до списку блокнотів користувача.

Актори: Користувач.

Опис: Цей сценарій описує процес створення нового блокнота для організації нотаток.

Основний хід подій:

1. Користувач відкриває список блокнотів.
2. Користувач натискає кнопку "Створити блокнот".
3. Система запитує назву для нового блокнота.
4. Користувач вводить назву блокнота та підтверджує дію.
5. Система створює блокнот та додає його до списку доступних блокнотів.

Винятки: Якщо блокнот із введеною назвою вже існує, система видає повідомлення про помилку.

Примітки: Відсутні.

Сценарій використання 3: Додавання нотаток у блокнот

Передумови: Користувач увійшов у систему, створив блокнот та хоча б одну нотатку.

Післяумови: Нотатка додана до вибраного блокнота.

Актори: Користувач.

Опис: Цей сценарій описує процес додавання існуючої нотатки до блокнота.

Основний хід подій:

1. Користувач відкриває список нотаток.
2. Користувач вибирає нотатку, яку потрібно додати до блокнота.
3. Користувач натискає кнопку "Додати до блокнота".
4. Система відкриває список доступних блокнотів.
5. Користувач вибирає блокнот та підтверджує вибір.
6. Система додає нотатку до вибраного блокнота та підтверджує успішне виконання операції.

Винятки:

- Якщо блокнот не вибрано, система видає повідомлення про помилку.
- Якщо нотатка вже є у вибраному блокноті, система інформує про це.

Примітки: Відсутні.

3. Схема класів

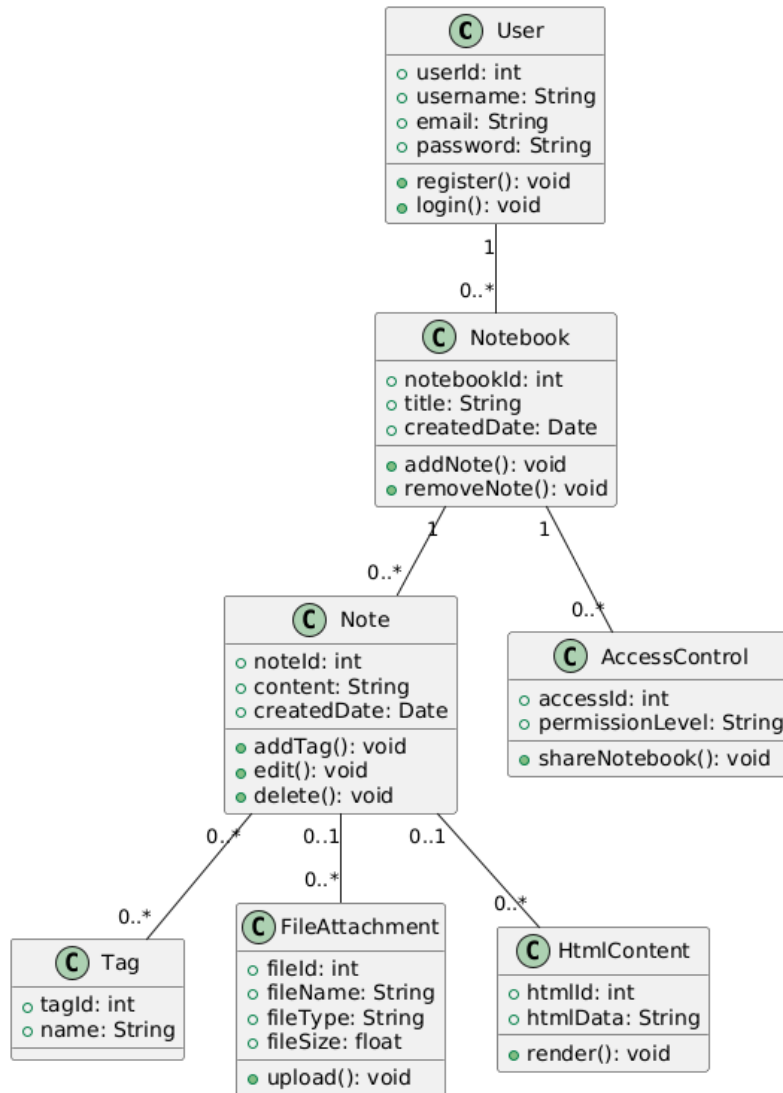


Рисунок 3-1 Діаграма класів

Основний клас — **User**, що відповідає за реєстрацію та авторизацію користувачів. Кожен користувач може мати кілька **Notebook**, які містять одну або більше **Note**. Нотатки можуть мати текстовий вміст, мітки (**Tag**), прикріплені файли (**FileAttachment**) або HTML-контент (**HtmlContent**). Клас **AccessControl** дозволяє ділитися блокнотами з іншими користувачами, задаючи рівні доступу (перегляд, редагування тощо).

4. Структура бази даних

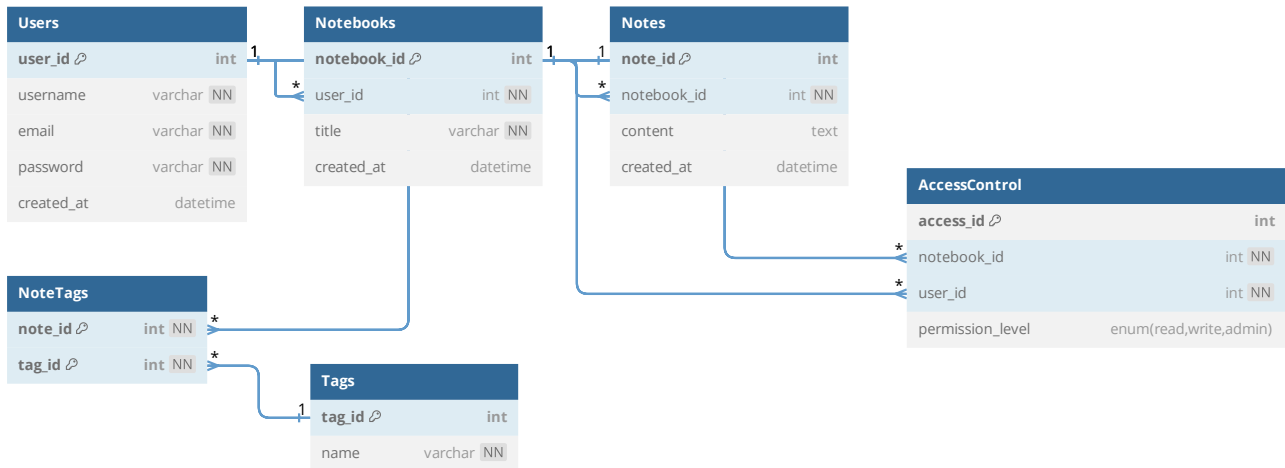


Рисунок 4-1 Діаграма бази даних

База даних (Рисунок 4) є реляційною і призначена для багатокористувацької системи ведення нотаток. Вона включає таблиці **Users**, **Notebooks**, **Notes**, **Tags**, **NoteTags**, та **AccessControl**, пов'язані між собою за допомогою відношень "один-до-багатьох" та "багато-до-багатьох". Таблиця **Users** зберігає інформацію про користувачів і має зв'язок "один-до-багатьох" із таблицею **Notebooks**, яка містить блокноти користувачів. Таблиця **Notes** пов'язана з **Notebooks** відношенням "один-до-багатьох", зберігаючи окремі нотатки. **Tags** організовують категорії для нотаток, а **NoteTags** реалізує зв'язок "багато-до-багатьох" між нотатками та тегами. Таблиця **AccessControl** дозволяє задавати права доступу до блокнотів із рівнями доступу ("read", "write", "admin"), створюючи зв'язок "багато-до-багатьох" між користувачами та блокнотами.

Висновки:

У результаті виконання лабораторної роботи було успішно створено діаграму прецедентів для обраної системи, побудовано діаграму класів та концептуальну модель системи. Це дозволило краще зрозуміти структуру та взаємодію між різними елементами системи, а також забезпечило можливість реалізації ключових класів для роботи з базою даних. UML-діаграми допомагають візуалізувати систему та полегшують процес її проектування та подальшої розробки.